

Bucles, funciones y reutilización de lógica en JavaScript

Enfoque de la sesión

En esta sesión vas a dar un paso importante: pasar de escribir instrucciones sueltas a **organizar el comportamiento del programa**. Para ello necesitas aprender a **repetir tareas, encapsular lógica y reutilizar código**. Estos tres elementos son fundamentales para escribir programas mantenibles y serán imprescindibles cuando más adelante trabajes con arrays, eventos, formularios, renderizado dinámico y React.

1. Introducción

En la sesión anterior trabajaste con variables, tipos, operadores y condicionales. Eso te permitió almacenar información y tomar decisiones básicas. Sin embargo, un programa real necesita algo más que evaluar una condición concreta: también necesita repetir acciones, agrupar instrucciones relacionadas y reutilizar la misma lógica en distintos contextos.

Ahí es donde entran en juego los bucles y las funciones.

Los **bucles** te permiten repetir una tarea mientras se cumpla una condición o mientras existan elementos que recorrer. Las **funciones** te permiten encapsular una parte del programa bajo un nombre, reutilizarla cuando lo necesites y evitar duplicación innecesaria.

Aprender estas dos ideas no consiste solo en memorizar sintaxis. Significa empezar a pensar en términos de:

- repetición controlada;
- modularización de lógica;
- claridad del flujo del programa;
- reutilización de comportamiento.

Esta sesión es especialmente importante porque sienta la base de cómo se estructura el código en JavaScript moderno. Más adelante, cuando construyas componentes, handlers, validaciones o funciones de transformación de datos, volverás constantemente a lo que vas a trabajar aquí.

2. Repetición de tareas: introducción a los bucles

2.1. Por qué necesitas bucles

Imagina que quieres mostrar cinco veces un mismo mensaje. Sin un bucle, tendrías que repetir la instrucción manualmente:

```
console.log("Hola");  
console.log("Hola");  
console.log("Hola");  
console.log("Hola");  
console.log("Hola");
```

Aunque este ejemplo es pequeño, el problema se vuelve evidente enseguida: repetir instrucciones a mano no escala bien y hace el código más difícil de mantener.

Un bucle te permite expresar esa repetición de forma clara y controlada.

Idea clave

Un bucle no sirve solo para “repetir por repetir”. Sirve para **automatizar una tarea repetitiva** y expresar mejor la intención del programa.

3. Bucle `for`

3.1. Estructura general

El bucle `for` es muy útil cuando sabes de antemano cuántas veces quieres repetir una acción.

```
for (let i = 0; i < 5; i++) {  
  console.log("Hola");  
}
```

3.2. Lectura paso a paso

La estructura del `for` tiene tres partes:

Parte	Función
<code>let i = 0</code>	inicializa una variable de control
<code>i < 5</code>	establece la condición de continuación

Parte	Función
<code>i++</code>	actualiza la variable en cada iteración

En este caso:

- el contador empieza en `0` ;
- el bloque se ejecuta mientras `i` sea menor que `5` ;
- en cada vuelta, `i` aumenta en una unidad.

3.3. Ejemplo con número de iteración

```
for (let i = 1; i <= 5; i++) {  
  console.log(`Iteración número ${i}`);  
}
```

3.4. Cuándo usar `for`

Conviene usar `for` cuando:

- conoces el número de repeticiones;
- necesitas trabajar con un índice;
- quieres controlar de forma precisa el inicio, el final y el paso.

3.5. Riesgos frecuentes

Error de límites

```
for (let i = 0; i <= 5; i++) {  
  console.log(i);  
}
```

Este bucle muestra seis valores: `0` , `1` , `2` , `3` , `4` , `5` .

Si esperabas cinco repeticiones, has cometido un error de límite.

Bucle infinito

```
for (let i = 0; i < 5;) {  
  console.log(i);  
}
```

Aquí falta la actualización de `i`, así que la condición nunca cambia y el bucle puede no terminar.

Referencias específicas del subtema

- MDN Web Docs — `for`
 - MDN Web Docs — Increment (`++`)
-

4. Bucle `while`

4.1. Idea general

El bucle `while` se utiliza cuando quieres repetir una acción **mientras una condición siga siendo verdadera**, pero no necesariamente sabes desde el principio cuántas iteraciones habrá.

```
let contador = 1;

while (contador <= 5) {
  console.log(`Valor actual: ${contador}`);
  contador++;
}
```

4.2. Diferencia respecto a `for`

`while` suele ser buena opción cuando:

- el número exacto de repeticiones no está tan claro;
- el final depende de una condición cambiante;
- quieres resaltar la condición como eje del bucle.

4.3. Riesgo principal

Si olvidas actualizar la variable o modificar la condición, puedes provocar un bucle infinito.

```
let contador = 1;

while (contador <= 5) {
  console.log(contador);
}
```

En este caso `contador` nunca cambia.

Precaución

Cada vez que escribas un `while`, pregúntate:
¿qué hace que esta condición deje de cumplirse?

Referencias específicas del subtema

- MDN Web Docs — `while`
-

5. Bucle `for...of`

5.1. Qué resuelve

`for...of` permite recorrer valores de una estructura iterable, como un array o una cadena, de una manera más clara que un `for` tradicional en muchos casos.

```
const nombres = ["Ana", "Luis", "Marta"];

for (const nombre of nombres) {
  console.log(nombre);
}
```

5.2. Ventajas

- el código es más legible;
- no necesitas manejar índices manualmente si no hacen falta;
- expresa muy bien la idea de “recorrer elementos”.

5.3. Ejemplo con cadena

```
const palabra = "Hola";

for (const letra of palabra) {
  console.log(letra);
}
```

5.4. Cuándo es una buena elección

Conviene usar `for...of` cuando:

- quieres recorrer los valores de una colección;
- no necesitas conocer el índice;
- priorizas legibilidad.

Referencias específicas del subtema

- MDN Web Docs — `for...of`

6. Comparación práctica entre `for`, `while` y `for...of`

Estructura	Mejor uso habitual	Idea principal
<code>for</code>	cuando controlas número de repeticiones o índice	repetir con control explícito
<code>while</code>	cuando dependes de una condición cambiante	repetir mientras algo siga siendo cierto
<code>for...of</code>	cuando recorres valores de una colección	iterar elementos de forma legible

Criterio de elección

No se trata de buscar “el bucle correcto universal”, sino el que expresa mejor la intención de tu código.

7. Funciones: encapsular y reutilizar comportamiento

7.1. Qué es una función

Una función es un bloque de código reutilizable que realiza una tarea concreta.

```
function saludar() {  
  console.log("Hola");  
}
```

Definir la función no la ejecuta automáticamente. Para usarla, tienes que llamarla:

```
function saludar() {  
  console.log("Hola");  
}  
  
saludar();
```

7.2. Por qué son importantes

Las funciones permiten:

- evitar repetir código;
- dar nombre a una tarea;
- dividir un problema grande en partes pequeñas;
- mejorar la legibilidad;
- reutilizar comportamiento.

Idea clave

Una buena función suele tener una responsabilidad clara y un nombre que describa bien lo que hace.

Referencias específicas del subtema

- MDN Web Docs — Functions
-

8. Funciones declaradas

8.1. Sintaxis

```
function mostrarMensaje() {  
  console.log("Bienvenido al sistema");  
}
```

8.2. Con parámetros

Los parámetros permiten que la función reciba información.

```
function saludar(nombre) {  
  console.log(`Hola, ${nombre}`);  
}
```

```
}  
  
saludar("Claudia");  
saludar("Javier");
```

8.3. Con retorno

Una función puede devolver un valor mediante `return`.

```
function sumar(a, b) {  
  return a + b;  
}  
  
const resultado = sumar(4, 6);  
console.log(resultado);
```

8.4. Diferencia entre mostrar y devolver

Observa la diferencia:

```
function sumarYMostrar(a, b) {  
  console.log(a + b);  
}  
  
function sumarYDevolver(a, b) {  
  return a + b;  
}
```

La primera imprime el resultado. La segunda lo devuelve para poder reutilizarlo.

```
const total = sumarYDevolver(3, 4);  
console.log(total * 2);
```

Esto no sería posible si la función solo mostrara el resultado sin devolverlo.

Criterio importante

Muchas veces conviene que una función **devuelva un valor** en lugar de limitarse a mostrarlo, porque así podrás reutilizar el resultado en otras partes del programa.

Referencias específicas del subtema

- MDN Web Docs — `return`
 - MDN Web Docs — Function parameters
-

9. Funciones expresadas

9.1. Definición

También puedes guardar una función dentro de una variable.

```
const saludar = function (nombre) {  
  console.log(`Hola, ${nombre}`);  
};
```

9.2. Cuándo aparecen

Las funciones expresadas son frecuentes cuando:

- una función se asigna a una constante o variable;
- se pasa como argumento a otra función;
- se trabaja con callbacks;
- se quiere tratar la función como un valor más.

Referencias específicas del subtema

- MDN Web Docs — Function expression
-

10. Arrow functions

10.1. Sintaxis básica

Las arrow functions son una forma más compacta de expresar funciones.

```
const saludar = (nombre) => {  
  console.log(`Hola, ${nombre}`);  
};
```

10.2. Forma abreviada con retorno implícito

Si la función tiene una sola expresión, puedes omitir llaves y `return`.

```
const duplicar = (numero) => numero * 2;

console.log(duplicar(5));
```

10.3. Cuándo resultan útiles

Las arrow functions se utilizan mucho en JavaScript moderno, especialmente cuando:

- necesitas funciones pequeñas;
- pasas funciones como argumento;
- trabajas con métodos de arrays;
- quieres una sintaxis más breve.

10.4. Precaución conceptual

Aunque visualmente parezcan solo una versión “más corta”, no son idénticas en todos los aspectos a las funciones tradicionales. Más adelante estudiarás diferencias importantes relacionadas con `this`, pero por ahora lo esencial es distinguir su sintaxis y sus usos básicos.

Referencias específicas del subtema

- MDN Web Docs — Arrow function expressions
-

11. Parámetros, argumentos y valores por defecto

11.1. Diferencia entre parámetro y argumento

- **Parámetro**: nombre que aparece en la definición de la función.
- **Argumento**: valor real que se pasa al llamar a la función.

```
function saludar(nombre) {
  console.log(`Hola, ${nombre}`);
}

saludar("Lucía");
```

Aquí `nombre` es el parámetro y `"Lucía"` es el argumento.

11.2. Valores por defecto

Puedes asignar un valor por defecto a un parámetro por si no se proporciona argumento.

```
function saludar(nombre = "invitado") {  
  console.log(`Hola, ${nombre}`);  
}  
  
saludar();  
saludar("Mónica");
```

11.3. Ventajas

Los valores por defecto:

- evitan algunos errores;
- hacen el comportamiento de la función más previsible;
- mejoran la robustez de la función.

Referencias específicas del subtema

- MDN Web Docs — Default parameters
-

12. Scope: ámbito de las variables

12.1. Qué significa el ámbito

El ámbito o *scope* determina desde dónde puede accederse a una variable.

12.2. Ámbito global

Una variable declarada fuera de cualquier función o bloque tiene alcance amplio.

```
const curso = "Bootcamp";  
  
function mostrarCurso() {  
  console.log(curso);  
}  
  
mostrarCurso();
```

12.3. Ámbito de función

Una variable declarada dentro de una función solo existe dentro de ella.

```
function crearMensaje() {  
  const mensaje = "Hola desde dentro";  
  console.log(mensaje);  
}  
  
crearMensaje();  
// console.log(mensaje); // Error
```

12.4. Ámbito de bloque

Las variables declaradas con `let` y `const` dentro de un bloque solo existen dentro de ese bloque.

```
if (true) {  
  const aviso = "Visible solo aquí";  
  console.log(aviso);  
}  
  
// console.log(aviso); // Error
```

12.5. Por qué importa

Entender el ámbito es esencial para:

- evitar errores de acceso;
- no sobrescribir valores sin querer;
- mantener el código más seguro y claro.

Idea clave

Una variable debe vivir en el ámbito más pequeño razonable.
Cuanto más limitada esté su visibilidad, más fácil será controlar su uso.

Referencias específicas del subtema

- MDN Web Docs — Scope
- MDN Web Docs — Block statement

13. Hoisting

13.1. Qué es

El hoisting describe el comportamiento por el cual ciertas declaraciones parecen “elevarse” conceptualmente dentro de su ámbito durante la fase de preparación del código.

13.2. Caso de funciones declaradas

Las funciones declaradas pueden invocarse antes de aparecer físicamente en el archivo.

```
saludar();

function saludar() {
  console.log("Hola");
}
```

13.3. Caso de `var`

Con `var`, la declaración existe antes de su línea de código, pero su valor inicial es `undefined`.

```
console.log(ciudad); // undefined
var ciudad = "Sevilla";
```

13.4. Caso de `let` y `const`

Con `let` y `const`, aunque la declaración exista en la fase previa, no puedes acceder a la variable antes de su declaración efectiva.

```
// console.log(nombre); // Error
const nombre = "Elena";
```

13.5. Criterio práctico

Aunque conocer hoisting es importante, no debes usarlo como estilo de escritura. Lo recomendable es:

- declarar variables antes de utilizarlas;
- definir el código en un orden legible;
- evitar confiar en efectos implícitos.

 **Recomendación de estilo**

Entender hoisting sirve para explicar comportamientos del lenguaje, no para justificar código confuso.

Referencias específicas del subtema

- MDN Web Docs — Hoisting
- MDN Web Docs — `var`
- MDN Web Docs — `let`

14. Callbacks: funciones que se pasan a otras funciones

14.1. Qué es un callback

Un callback es una función que se pasa como argumento a otra función para que esta la ejecute en un momento determinado.

```
function ejecutarOperacion(operacion) {  
  operacion();  
}  
  
function mostrarSaludo() {  
  console.log("Hola desde el callback");  
}  
  
ejecutarOperacion(mostrarSaludo);
```

14.2. Por qué son importantes

Los callbacks aparecen constantemente en JavaScript:

- en eventos;
- en asincronía;
- en métodos de arrays;
- en funciones reutilizables.

14.3. Ejemplo con función anónima

```
function ejecutarOperacion(operacion) {  
  operacion();  
}
```

```
}

ejecutarOperacion(function () {
  console.log("Esto también es un callback");
});
```

14.4. Ejemplo con arrow function

```
function ejecutarOperacion(operacion) {
  operacion();
}

ejecutarOperacion(() => {
  console.log("Callback con arrow function");
});
```

Referencias específicas del subtema

- MDN Web Docs — First-class Function
 - MDN Web Docs — Callback function
-

15. Closures: recordar el entorno donde una función fue creada

15.1. Idea general

Una closure aparece cuando una función conserva acceso a variables de su entorno incluso después de que ese entorno haya terminado de ejecutarse.

Puede parecer abstracto al principio, así que conviene verlo con un ejemplo.

```
function crearContador() {
  let contador = 0;

  return function () {
    contador++;
    return contador;
  };
}

const contar = crearContador();
```

```
console.log(contar()); // 1
console.log(contar()); // 2
console.log(contar()); // 3
```

15.2. Qué está ocurriendo

- `crearContador()` crea una variable local llamada `contador` ;
- devuelve una función interna;
- esa función interna sigue teniendo acceso a `contador` ;
- cada vez que se invoca, puede modificar y reutilizar ese valor.

15.3. Por qué es útil

Las closures permiten:

- mantener estado encapsulado;
- evitar exponer variables innecesarias;
- construir utilidades reutilizables;
- comprender mejor cómo JavaScript gestiona el contexto léxico.

Nivel de exigencia en esta sesión

No necesitas dominar closures con complejidad avanzada todavía. Lo importante es comprender que una función puede “recordar” variables del lugar donde fue creada.

Referencias específicas del subtema

- MDN Web Docs — Closures

16. Diseño inicial de funciones: claridad, responsabilidad y reutilización

16.1. Una función no debería hacer demasiadas cosas

Cuando una función:

- valida datos;
- calcula resultados;
- muestra mensajes;

- cambia variables externas;
- y además imprime por consola;

es fácil que se vuelva difícil de mantener.

16.2. Responsabilidad clara

Conviene que una función responda a una intención concreta.

Menos clara

```
function procesarCompra(precio, descuento) {  
  if (precio > 0) {  
    const total = precio - descuento;  
    console.log(`Total: ${total}`);  
  } else {  
    console.log("Precio no válido");  
  }  
}
```

Más separada

```
function calcularTotal(precio, descuento) {  
  return precio - descuento;  
}  
  
function esPrecioValido(precio) {  
  return precio > 0;  
}
```

16.3. Por qué importa

Diseñar funciones más pequeñas y específicas ayuda a:

- entender mejor el código;
- reutilizar lógica;
- detectar errores;
- preparar el código para pruebas y transformaciones posteriores.

 **Idea clave**

Reutilizar código no es solo “meter cosas en una función”.

También implica decidir **qué responsabilidad exacta** debe tener esa función.

Referencias específicas del subtema

- MDN Web Docs — Functions
 - MDN Web Docs — Function parameters
-

17. Análisis técnico avanzado

17.1. Repetir no significa duplicar

Hay una diferencia importante entre:

- repetir una **acción** mediante un bucle;
- repetir **código copiado** varias veces.

El primer caso automatiza una tarea. El segundo genera mantenimiento innecesario.

17.2. Una función demasiado grande deja de ayudar

No basta con “usar funciones”. También importa **cómo** se diseñan. Si una función:

- hace demasiadas cosas;
- tiene demasiados parámetros;
- mezcla cálculo, validación y salida por consola;
- es difícil de entender por su nombre o su longitud;

entonces deja de aportar claridad.

17.3. Scope como herramienta de control

El ámbito no es solo una restricción técnica. También es una ayuda para diseñar mejor. Limitar dónde vive una variable te obliga a estructurar mejor el programa y reduce errores accidentales.

17.4. Hoisting no debe marcar tu estilo de escritura

Saber que una función declarada puede usarse antes de su definición no significa que debas escribir el código en desorden. La legibilidad sigue siendo prioritaria.

17.5. Closures como base de ideas posteriores

Aunque en esta sesión las trabajes de forma sencilla, las closures están detrás de muchos patrones importantes de JavaScript. Comprenderlas poco a poco te ayudará más adelante a entender mejor callbacks, asincronía y ciertos comportamientos de React.

Referencias específicas del subtema

- [MDN Web Docs — Functions](#)
- [MDN Web Docs — Closures](#)
- [MDN Web Docs — Scope](#)

18. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```
function generarResumenNumeros(limite) {
  let sumaTotal = 0;

  for (let numero = 1; numero <= limite; numero++) {
    sumaTotal += numero;
  }

  return sumaTotal;
}

function mostrarResultado(limite, callback) {
  const resultado = generarResumenNumeros(limite);
  callback(resultado);
}

mostrarResultado(5, (resultado) => {
  console.log(`La suma de los números del 1 al 5 es ${resultado}`);
});
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
Bucle <code>for</code>	repetición controlada desde 1 hasta el límite
Función declarada	encapsulación de la lógica de suma

Elemento	Uso en el ejemplo
Retorno	devolución del valor calculado
Callback	tratamiento posterior del resultado
Scope	<code>sumaTotal</code> y <code>numero</code> existen dentro de su ámbito correspondiente
Arrow function	callback breve y legible

19. Errores frecuentes al comenzar

⚠ Errores muy habituales

Detectar estos fallos pronto te ayudará a escribir funciones y bucles más seguros y claros.

19.1. Repetir manualmente lo que debería resolverse con un bucle

```
console.log("Producto 1");  
console.log("Producto 2");  
console.log("Producto 3");  
console.log("Producto 4");
```

Cuando la estructura es repetitiva, conviene pensar si un bucle expresa mejor la intención.

19.2. Crear bucles infinitos por no actualizar la condición

```
let numero = 1;  
  
while (numero <= 5) {  
  console.log(numero);  
}
```

Aquí falta modificar `numero`.

19.3. Escribir funciones sin retorno cuando luego necesitas reutilizar el valor

```
function sumar(a, b) {  
  console.log(a + b);  
}
```

```
}

const resultado = sumar(2, 3);
console.log(resultado); // undefined
```

19.4. Utilizar variables fuera de su ámbito

```
function crearUsuario() {
  const nombre = "Alicia";
}

// console.log(nombre); // Error
```

19.5. Elegir nombres poco expresivos

```
function x(a, b) {
  return a + b;
}
```

Ese código puede funcionar, pero cuesta mucho más entenderlo que una versión con nombres significativos.

```
function calcularTotal(precioBase, recargo) {
  return precioBase + recargo;
}
```

19.6. Mezclar demasiadas responsabilidades en una única función

Una función que valida, calcula, muestra mensajes y modifica datos a la vez suele ser más difícil de mantener.

20. Síntesis de la sesión

En esta sesión has trabajado dos pilares fundamentales del desarrollo en JavaScript:

- la repetición controlada mediante bucles;
- la encapsulación y reutilización de comportamiento mediante funciones.

También has comenzado a entender conceptos esenciales para escribir código más sólido:

- parámetros y retorno;

- scope;
- hoisting;
- callbacks;
- closures básicas;
- primeros criterios de diseño de funciones.

Todo esto será imprescindible en las siguientes sesiones. Muy pronto necesitarás recorrer colecciones, transformar datos, responder a eventos o dividir tu código en piezas reutilizables. Ninguna de esas tareas se entiende bien sin esta base.

✓ **Al terminar esta sesión deberías ser capaz de...**

- utilizar `for`, `while` y `for...of` con criterio;
- crear funciones declaradas, expresadas y flecha;
- diferenciar parámetros y argumentos;
- devolver resultados con `return`;
- comprender el ámbito de las variables;
- reconocer el comportamiento básico del hoisting;
- entender qué es un callback;
- interpretar una closure sencilla;
- empezar a diseñar funciones más claras y reutilizables.

21. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — Functions
- MDN Web Docs — Function expression
- MDN Web Docs — Arrow function expressions
- MDN Web Docs — Default parameters
- MDN Web Docs — `return`
- MDN Web Docs — `for`
- MDN Web Docs — `while`
- MDN Web Docs — `for...of`
- MDN Web Docs — Scope
- MDN Web Docs — Closures

- MDN Web Docs — Hoisting

Recomendación de estudio

Al repasar esta sesión, no te limites a leer ejemplos. Cambia los valores, altera las condiciones de salida de los bucles, añade parámetros nuevos a las funciones y prueba qué ocurre si una función devuelve un valor o solo lo muestra por consola. Entender JavaScript exige observar cómo cambia el comportamiento del programa cuando modificas su estructura.